Express.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 Your First Server and Routes

03 Request and Response Objects

04 Middleware

05 Routers and Project Structure

06 Serving Static Files and Templates

Intermediate (6 topics)

07 REST API Design

08 Validation

09 Error Handling Middleware

10 Connecting to a Database

11 Environment Configuration

12 CORS and Cookies

Advanced (6 topics)

13 Authentication and Authorization

14 File Uploads

15 Security Hardening

16 Logging and Monitoring

17 Testing APIs

18 API Documentation

Master (4 topics)

19	Layered Architecture
20	Caching, Queues and Performance
21	Deployment with Docker and CI/CD
22	Real Project: Task/Blog/E-commerce API

Beginner Level

01. Introduction and Setup

Express is a minimal web framework for Node.js for building APIs and web apps.

```
npm i express
import express from 'express';
const app = express();
app.listen(3000);
```

02. Your First Server and Routes

Define routes with `app.get/post/put/delete(path, handler)`.

```
app.get('/', (req, res) => res.send('Hello Express'));
```

03. Request and Response Objects

`req.params`, `req.query`, `req.body`, `req.headers`; `res.json`, `res.status`, `res.redirect`.

```
app.get('/users/:id', (req, res) => res.json({ id: req.params.id, q: req.query.q }));
```

04. Middleware

Functions `(req, res, next)` run in order. Built-ins: `express.json()`, `express.static()`.

```
app.use(express.json());
app.use((req, res, next) => { console.log(req.method, req.url); next(); });
```

05. Routers and Project Structure

Group routes with `express.Router()` and mount them under prefixes.

```
const router = express.Router();
router.get('/', listUsers);
app.use('/api/users', router);
```

06. Serving Static Files and Templates

Serve assets with `express.static` and render views with EJS/Pug.

```
app.use(express.static('public'));
app.set('view engine', 'ejs');
```

Intermediate Level

07. REST API Design

Use nouns for resources, correct verbs and status codes (200, 201, 400, 401, 404, 500), pagination and versioning.

```
app.post('/api/todos', (req, res) => res.status(201).json(todo));
```

08. Validation

Never trust input; validate bodies with Zod, Joi or express-validator.

```
const schema = z.object({ title: z.string().min(1) });
const data = schema.parse(req.body);
```

09. Error Handling Middleware

Four-argument middleware `(err, req, res, next)` centralises errors; wrap async handlers.

```
app.use((err, req, res, next) => res.status(err.status ?? 500).json({ error: err.message }));
```

10. Connecting to a Database

Use Mongoose (MongoDB), `pg` / `mysql2` or Prisma; keep data access in repository/service layers.

```
const user = await prisma.user.findUnique({ where: { id: Number(req.params.id) } });
```

11. Environment Configuration

Use environment variables for ports, URLs and secrets; one config module.

```
const port = process.env.PORT ?? 3000;
```

12. CORS and Cookies

Allow browser apps from other origins with `cors`; use `cookie-parser` and secure cookie flags.

```
app.use(cors({ origin: 'https://myapp.com', credentials: true }));
```

Advanced Level

13. Authentication and Authorization

Register/login with hashed passwords, JWT or sessions, `authenticate` middleware and role checks.

```
function auth(req, res, next) {
  const t = req.headers.authorization?.split(' ')[1];
  try { req.user = jwt.verify(t, SECRET); next(); } catch { res.sendStatus(401); }
}
```

14. File Uploads

Handle multipart forms with Multer; validate type and size; store on S3 for production.

```
const upload = multer({ dest: 'uploads/', limits: { fileSize: 2e6 } });
app.post('/avatar', upload.single('file'), handler);
```

15. Security Hardening

Helmet, rate limiting, input sanitising, HTTPS, CSRF protection for cookies, secure headers.

```
app.use(helmet());
app.use(rateLimit({ windowMs: 60_000, limit: 100 }));
```

16. Logging and Monitoring

Request logging with morgan/pino, correlation ids and health endpoints.

```
app.use(morgan('combined'));
app.get('/health', (req, res) => res.json({ ok: true }));
```

17. Testing APIs

Test routes with Supertest and Vitest/Jest; use a test database or mocks.

```
const res = await request(app).get('/api/todos');
expect(res.status).toBe(200);
```

18. API Documentation

Document with OpenAPI/Swagger UI so frontend teams can try endpoints.

```
app.use('/docs', swaggerUi.serve, swaggerUi.setup(spec));
```

Master Level

19. Layered Architecture

Routes -> controllers -> services -> repositories, with dependency injection and DTO validation.

```
controllers/ services/ repositories/ middlewares/ routes/
```

20. Caching, Queues and Performance

Cache with Redis, compress responses, paginate, use background jobs, cluster mode.

```
app.use(compression());
```

21. Deployment with Docker and CI/CD

Containerise, use environment secrets, reverse proxy with Nginx, automated tests and deploys.

```
docker build -t api . && docker run -p 3000:3000 api
```

22. Real Project: Task/Blog/E-commerce API

Full REST API with auth, roles, validation, database, tests, docs and deployment.

Interview Question Bank

Q1. What is Express?
A minimal, flexible Node.js web framework providing routing and middleware.

Q2. What is middleware?
A function with `(req, res, next)` that can modify the request/response or end the cycle.

Q3. How does error handling work?
Error-handling middleware has four arguments and must be registered after routes.

Q4. `app.use` vs `app.get`?
`use` mounts middleware for all methods/paths (prefix); `get` handles only GET for a path.

Q5. req.params vs req.query vs req.body?
Route segments, URL query string, and parsed request payload.

Q6. How do you secure an Express API?
Helmet, validation, auth (JWT), rate limiting, CORS configuration, HTTPS.

Q7. What is CORS?
A browser mechanism restricting cross-origin requests; the server opts in with headers.

Q8. How do you structure a large app?
Routers, controllers, services and repositories with config and middleware folders.

Q9. How do you handle async errors?
Use `async` handlers with a wrapper or Express 5, which forwards rejected promises to error middleware.

Q10. PUT vs PATCH?
PUT replaces a resource; PATCH changes part of it.

Capstone Project Ideas

1. Todo REST API
2. Blog API with auth
3. Library management API
4. E-commerce backend
5. URL shortener